Vibe Coding prompt

Act as a Solidity and Python Web3 developer. Build verify withdrawal controls against reentrancy for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: Checks-effects-interactions plus entered guard prevent recursive withdrawal; an actual adversarial contract probes the boundary. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy actual EVM bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

Review prompt

Review every external function for authorization, state transitions, integer units and invariants. Point to exact lines. Produce a requirement-to-assertion matrix and add a failing test before repairing any defect.

Scenario-specific generation constraints

Depositor account0 deposits1ETH. Deploy ReenterProbe with vault address. Account1 funds attack0.1ETH; receive callback attempts vault.withdraw recursively and records blocked=true. Read probe balance entry0 and vault balance1ETH. Withdraw original depositor; vault ends zero.

Expected assertions: ReenterProbe.blocked=true; victim deposit preserved1ETH; no duplicate payout.

Extension prompt

Temporarily remove guard and zeroing in a separate vulnerable copy, demonstrate theft only in this isolated chain, then restore controls. Return the exact changed source lines and positive/negative tests.